

Vcpkg+CMake+Vscode Setup in MacOS

There are 2 ways to setup Vcpkg.

Globally

Install vcpkg

```
cd
git clone https://github.com/Microsoft/vcpkg.git
~/vcpkg/bootstrap-vcpkg.sh
~/vcpkg/vcpkg integrate install
```

Setup environment variables

Edit ~/.bash_profile, add

```
export VCPKG_ROOT="$HOME/vcpkg"
export PATH="$PATH:$VCPKG_ROOT"
```

Now, you can execute `vcpkg install` everywhere, and execute `vcpkg install <pkg>` will install the package into `$VCPKG_ROOT/packages/`.

Execute cmake

You should define `CMAKE_TOOLCHAIN_FILE` and `CMAKE_PREFIX_PATH`[^1] to run `cmake`.

```
cmake -
DCMAKE_TOOLCHAIN_FILE="$VCPKG_ROOT/scripts/buildsystems/vcpkg.cmake" -
DCMAKE_PREFIX_PATH="$VCPKG_ROOT/packages" -B build -S
```

Or you could just put `CMAKE_TOOLCHAIN_FILE` and `CMAKE_PREFIX_PATH` in `~/.bash_profile`

Setup vscode

You need to tell vscode how to `intellisence` and build.

1. intellisence & lint

That means you must tell vscode how to find a header files. It is done already by `vcpkg integrate install`. After integration, `vscode cpptools` add `vcpkg` headers automatically in `"/c_cpp_properties.json"`. like:

One include path per line.

```
${workspaceFolder}/**
${vcpkgRoot}/x64-osx/include
```

So vscode could find all the headers installed by *vcpkg*.

2. build

If you have already defined the environment variables `CMAKE_TOOLCHAIN_FILE` and `CMAKE_PREFIX_PATH`, just skip, or you could add setting in vscode:

```
{
  "cmake.configureSettings": {
    "CMAKE_TOOLCHAIN_FILE":
      "${userHome}/vcpkg/scripts/buildsystems/vcpkg.cmake"
  }
}
```

Local in project

If you want let the project manage itself, which is better, you could install *vcpkg* in project.

Install locally

```
git submodule add -b master https://github.com/Microsoft/vcpkg.git
./vcpkg/vcpkg/bootstrap-vcpkg.sh
```

Setup Vscode

There're 3 ways to setup `CMAKE_TOOLCHAIN_FILE` for vscode.

1. add following commands in `CMakeLists.txt`

```
set(CMAKE_TOOLCHAIN_FILE
    "${CMAKE_CURRENT_SOURCE_DIR}/vcpkg/scripts/buildsystems/vcpkg.cmake")
```

2. define `CMAKE_TOOLCHAIN_FILE` in `CMakePresets.json`^[^2], like:

```
"cacheVariables": {
  "CMAKE_TOOLCHAIN_FILE":
    "${sourceDir}/vcpkg/scripts/buildsystems/vcpkg.cmake",
}
```

3. define `CMAKE_TOOLCHAIN_FILE` in workspace settings.

```
{
  "cmake.configureSettings": {
    "CMAKE_TOOLCHAIN_FILE":
      "${sourceDir}/vcpkg/scripts/buildsystems/vcpkg.cmake"
  }
}
```

I prefer the first way, since it is explicit.

Resources

- [Cross-Platform Pitfalls and How to Avoid Them](#)
- [Get started with vcpkg](#)
- [Configure and build with CMake Presets in Visual Studio Code](#)

NOTES

- don't use extension *code runner* to run current source file any more, it's not worth setting `-I` and `-L` flags.

[^1]: `CMAKE_PREFIX_PATH` is needed only for the first time, and I don't know why.

[^2]: You should add `CMakePresets.json` and restart vscode to enable the usage of presets, since `cmake.useCMakePresets` is **auto** by default.